

| STAT214 Chapter 11 Binary Responses and Logistic Regression

| 1 Logistic Regression

| 1.1 Logistic Regression

- Since linear combinations produce unbounded and continuous predictions, instead of directly predicting the binary response itself, one of the most common approaches to generating linear combination-based predictions of binary responses is to instead predict the probability that the response will equal 1
- to produce a linear combination-based prediction of the binary response probability, we want each side of equation to either both be bounded between 0 or 1 or both be unbounded
- The most common such transformation is the **logistic** transformation, and the algorithm that uses a **logistic** linear combination to predict the **probability** of a success is called **logistic regression**

| 1.2 Definition: Odds (Ratio)

Odds (ratio) is defined as the ratio of the probability of a success p and the probability of a failure $1 - p$

$$\text{odds} = \frac{p}{1 - p}$$

| 1.3 Definition: Log Odds (Logit)

It is common to transform the odds to make them unbounded by applying a logarithmic transformation to the odds

$$\log \text{ odds} = \text{logit}(p) = \log \left(\frac{p}{1 - p} \right)$$

| 1.4 Definition: Logistic Regression

Logistic regression computes binary response probability predictions, p , based on the logistic-transformed linear combination:

$$p = \frac{1}{1 + e^{-(b_0 + b_1 x)}}$$

where x is a relevant predictive feature.

| 1.5 Generating Binary Response Predictions for Logistic Regression

A response **probability** prediction can be easily converted to a binary response prediction using a thresholding rule, such as

- Predict “1” (that the session will end with a purchase) if the predicted success probability, p , is **0.5 or higher**.
- Predict “0” (that the session will **not** end with a purchase) if the predicted success probability, p , is **lower than 0.5**.

It often turns out that a different threshold value will generate more accurate predictions when the two classes are not **balanced**

| 1.6 Definition: The Logistic Loss Function

The **logistic loss** function, which adds up the loss for each training set observation, is given by

$$\sum_{i \text{ in pos class}} (-\log p_i) + \sum_{i \text{ in neg class}} (-\log(1 - p_i))$$

approximately optimal values of b_0 and b_1 are typically chosen using a technique called **iteratively reweighted least squares (IWLS)**.

| 1.7 Logistic Regression with Multiple Predictive Features

Categorical variables need to be converted to a numeric format or to one-hot encoded (dummy) variables to be included in a logistic regression fit

| 1.8 Interpreting and Comparing the Coefficients

The coefficients of a logistic fit can be interpreted using the **log odds** as the amount that the **predicted** log odds of the positive event would increase if the value of the relevant feature were to increase by 1 unit, *without changing the value of the other features*. Recall that since this corresponds to a relationship between the **predicted** response and the predictive features, this interpretation does **not** convey any causal relationship between the actual observed response and the predictive features.

| 2 Quantitative Measures of Binary Predictive Performance

| 2.1 Prediction Accuracy and Prediction Error

Since we don't know the "true" response probabilities, we typically evaluate the (thresholded) **binary** response predictions, rather than the response probability predictions themselves.

- **prediction accuracy**: the proportion of observations whose binary response is **correctly** predicted

$$\text{prediction accuracy} = \frac{\text{number of correct predictions}}{n}$$

- **prediction error**: the proportion of observations whose binary response is **incorrectly** predicted

$$\text{prediction error} = \frac{\text{number of incorrect predictions}}{n}$$

⚠ Remark: Training Accuracy and Training Error ▾

When the prediction accuracy and the prediction error are computed on the training data, they are called the **training accuracy** and **training error**, respectively. However, since training error and training accuracy aren't particularly good indications of how well our algorithm will be able to predict the responses for external or future data

| 2.2 The Confusion Matrix

A common way to present the results of a predictive algorithm is to cross-tabulate the **observed** response labels with the **predicted** response labels in a 2-by-2 table called the **confusion matrix**

		Predicted	
		positive	negative
Observed	positive	# true pos ✓	# false neg ✗
	negative	# false pos ✗	# true neg ✓

The confusion matrix tabulates the number of the following:

- **True positive**: A positive class observation that is correctly predicted to be in the positive class
- **False positive**: A negative class observation that is incorrectly predicted to be in the positive class
- **True negative**: A negative class observation that is correctly predicted to be in the negative class
- **False negative**: A positive class observation that is incorrectly predicted to be in the negative class

| 2.3 True Positive Rate

The **true positive rate** corresponds to the proportion of the sessions in the positive class whose binary response is correctly predicted:

$$\begin{aligned}\text{true positive rate} &= \frac{(\text{number of correctly predicted positive class obs})}{\text{number of positive class observations}} \\ &= \frac{(\text{number of true positives})}{\text{number of positive class observations}}\end{aligned}$$

The true positive rate is often called the **sensitivity** or the **recall**

2.4 True Negative Rate

The **true negative rate** corresponds to the proportion of negative class sessions whose response/class is correctly predicted. The true negative rate is often called the **specificity**.

2.5 The Sensitivity-Specificity Trade-Off

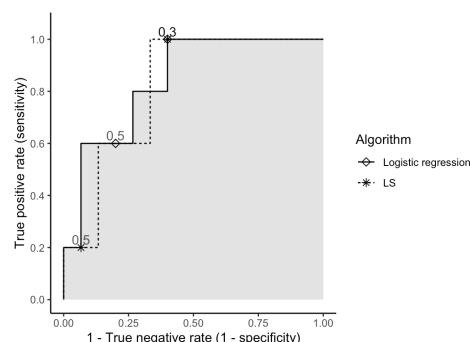
- **Increasing** the threshold value means that more of our predictions will be negative and fewer will be positive, which typically increases the true negative rate and decreases the true positive rate.
- **decreasing** the threshold value tends to decrease the true negative rate and increase the true positive rate.

2.6 Choose the Binary Threshold Value

- cross-validation (CV)
- However, recall that to tune a parameter, you need to have a measure that you're trying to optimize
- For prediction problems, many people will simply try to optimize the prediction accuracy

2.7 ROC Curves

- The ROC curve presents the potential performance of the algorithm across all possible predicted class probability thresholds.
- ROC curves can be used to choose the threshold that yields the best performance, guided by what kind of performance is being prioritized for your specific problem, and they can also be used to compare the performance of different algorithms.
- The algorithm whose ROC curve sits highest in the space (i.e., generally is closest to the top-left corner), has the best predictive performance potential (but the actual predictive performance will vary with the choice of threshold).



2.8 Class Imbalance

- For prediction problems where the data has a **class imbalance**, most algorithms will tend to overpredict the majority class
- Our preferred method of addressing class imbalance is to tweak the probability threshold, setting it equal to the proportion of observations in the positive class
- Other ways of dealing with class imbalance include downsampling the majority class ([Kubat and Matwin 1997](#)), upsampling the minority class, and the Synthetic Minority Oversampling Technique (SMOTE) ([Chawla et al. 2002](#)).

3 PCS Scrutinization of Binary Prediction Results

3.1 Predictability

On the **full** corresponding validation dataset, for each algorithm, evaluate

- confusion matrix
- accuracy

- true positive rate
- true negative rate
- AUC
- ROC curve
- the distribution of predictions for each class

3.2 Stability

3.2.1 Stability to Data Perturbations

Investigating how our predictive performance changes when we perturb our training data by

- adding random noise
- performing bootstrapping

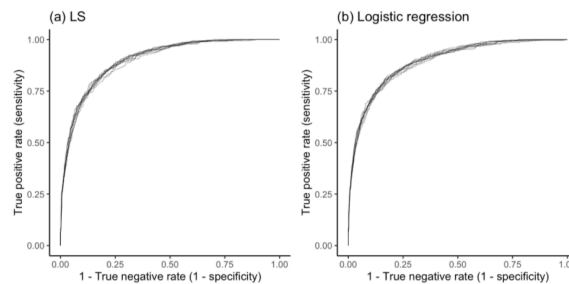


Figure 11.14: ROC curves for 10 versions of (a) the LS algorithm and (b) the logistic regression algorithms, each trained on a different random bootstrapped sample of the training data and evaluated on the validation data.

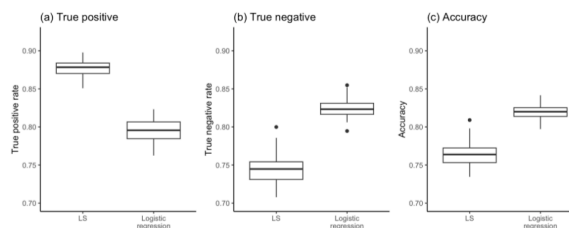


Figure 11.15: The distribution of the validation set (a) true positive rate (sensitivity), (b) true negative rate (specificity), and (c) accuracy metrics for the LS and logistic regression algorithms, each trained using 100 different data-perturbed versions of the training data. The binary prediction threshold value equals the proportion of (perturbed/bootstrapped) training points in the positive class.

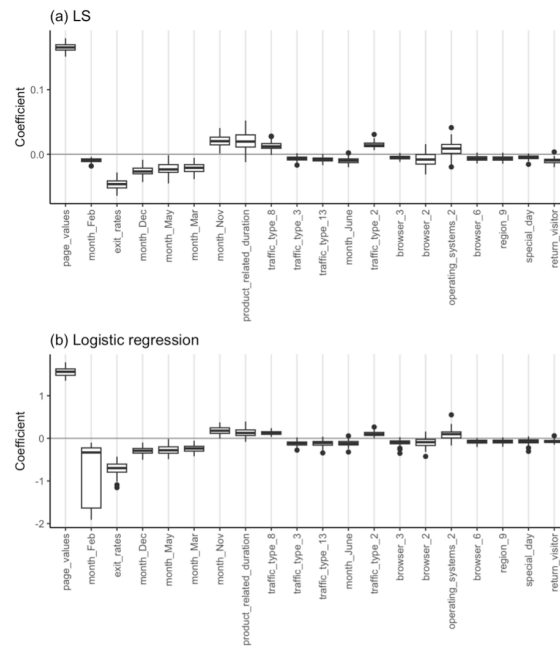


Figure 11.16: Boxplots showing the distribution of the coefficients from the (a) LS and (b) logistic regression fits across 100 perturbed (bootstrapped) versions of the training data. The variables have been standardized to a common scale before training the algorithms so the coefficient values are comparable. Note that we are only showing the 20 largest (in absolute value) coefficients based on the logistic regression algorithm.

3.2.2 Stability to Data Cleaning and Preprocessing Judgment Calls

- Create several alternative versions of the cleaned/preprocessed shopping dataset
- Before evaluating the stability of the LS and logistic regression algorithms to these judgment calls, we conducted predictability screening, where we include only the judgment call combinations that yield a validation set predictive performance that is above a certain threshold

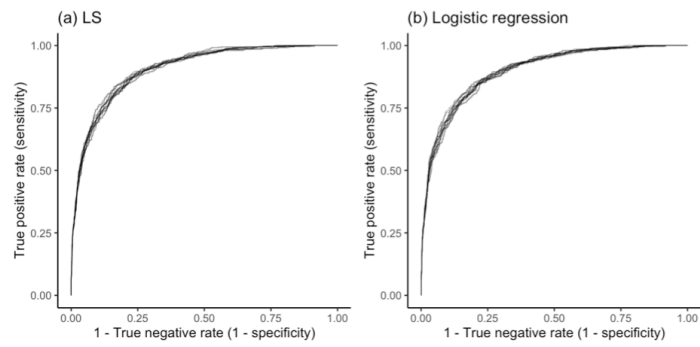


Figure 11.17: ROC curves for the (a) LS and (b) logistic regression algorithms based on the 16 judgment call-perturbed versions of the training datasets, each assessed on the relevant judgment call-perturbed version of the validation dataset.

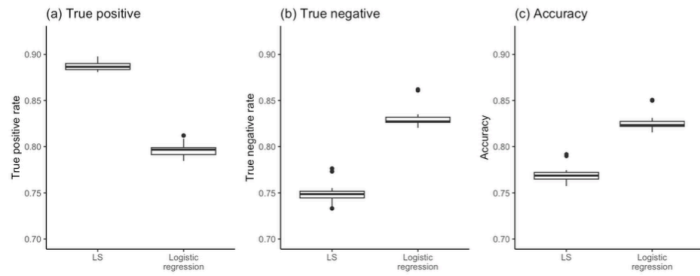


Figure 11.18: The distribution of the validation set (a) true positive rate, (b) true negative rate, and (c) accuracy metrics based on each of the judgment call-perturbed versions of the LS and logistic regression algorithms, where we are using a prediction threshold of 0.161 to convert the response predictions into a binary prediction for both algorithms.